Axona Topic IDs

How topics are addressed in `@axona/protocol` v3.1.0 — what a topic ID is, the two kinds (open and owned), and how you construct and use them in code.

Companion documents:

- Quick Start — 5-minute working roundtrip.
- API Reference — every public symbol.
- Programmer Guide — mental model + worked example.

> The Quick Start / API Reference / Programmer Guide above are still on the pre-v3 (`v2.40.0`) pub/sub surface (string topics + `publishId`). This note reflects the **current** v3.x addressing model (structured topic descriptors + `signWith`); where they disagree, this note wins.

---

1. A topic ID is a hash of a descriptor

A topic is not a bare string. It's a small **descriptor**, and the topic ID is a hash of that descriptor with a one-byte region prefix:

```
topicId = regionByte || SHA-256(canonical({ owner, name, write }))
```

Field	Meaning
`region`	a real geographic cell; becomes the leading byte of the ID
`owner`	an **Author ID** (a publish key), or absent
`name`	the human label — `"lobby"`, `"profile"`
`write`	the policy: `"open"` (default) or `"owner"`

Because the ID is a pure function of those fields, **anyone who knows the fields computes the identical ID** — no registry, no coordination. `owner` and `write` are *inside* the hash, so changing either yields a completely different topic.

You almost never handle the raw ID: you pass the **descriptor** to `peer.pub` / `peer.sub` and the kernel derives the ID. When you do want the hex string (to store or share compactly), call `deriveTopicId(descriptor)` — the same function the peer uses internally.

The `region` field: name or code

`region` accepts a region **name**, a **hex string**, a **decimal string**, or a **number** — all normalize to the same region byte:

```
{ region: 'useast', name: 'lobby' }   // name
{ region: '0x89',   name: 'lobby' }   // hex string  --\  all three resolve to the same
{ region: 137,      name: 'lobby' }   // number      --/  topic ID (leading byte 0x89)
```

`'useast'` = `'0x89'` = `137`. Unknown regions throw `RangeError`. The descriptor stored in the signed envelope always carries the numeric code, so a publisher using `0x89` and a subscriber using `'useast'` land on the same topic.

If you omit `region`, it defaults to the **publisher's own node region** (the top byte of its node ID). It is *never* derived from the author key. Consequence: two peers in the same region who both omit `region` meet automatically; to share a topic **across** regions, name the region explicitly so everyone computes the same ID.

---

2. Open topics — `{ region, name }`

The owner is absent and `write` defaults to `"open"`. Anyone can compute the ID, anyone can publish (each message self-signed by the author you choose, or anonymous), and anyone can subscribe.

```
import { createAuthorIdentity } from '@axona/protocol';

const me = await createAuthorIdentity();                    // a publish keypair

// publish - everyone using this region+name lands on the same topic
await peer.pub({ region: 'useast', name: 'lobby' }, 'hello', { signWith: me });

// subscribe - identical descriptor → identical ID
await peer.sub({ region: 'useast', name: 'lobby' }, (env) => {
  console.log(env.signerPubkey, env.message);               // signer (or undefined if anon) + payload
}, { since: 'all' });
```

Use open topics for lobbies, rooms, channels — anything where the name itself is the rendezvous and anyone may speak.

---

3. Owned topics — `{ region, owner, name, write: 'owner' }`

Fold **your Author ID** in as `owner` and set `write: 'owner'`. Now the topic ID incorporates your key, and **only messages signed by the owner key are accepted** — enforced independently at every node that stores the topic (it recomputes the ID from the *signed* descriptor and rejects anything where `signerPubkey !== owner`, throwing `WRITE_POLICY_VIOLATION`). So a profile or feed can't be spoofed even by someone talking directly to a storage node.

```
const me   = await createAuthorIdentity({ persistAs: 'me' });   // durable owner key
const feed = { region: 'useast', owner: me.authorId, name: 'profile', write: 'owner' };

await peer.pub(feed, { status: 'online' }, { signWith: me });   // only `me` may write
```

Reading stays open — but the ID is *deterministic, not secret.* What a stranger lacks is the owner's Author ID (a public key not listed in any directory), so they can't form the descriptor. In practice the owner gives subscribers what they need: the descriptor, or the precomputed ID.

```
// a subscriber the owner shared the feed with - must pass ALL the hashed fields
// (region, owner, name, write), since the ID is the hash of them:
await peer.sub(
  { region: 'useast', owner: aliceAuthorId, name: 'profile', write: 'owner' },
  onMsg, { since: 'all' }
);

// or share a compact ID instead of the descriptor:
const idHex = await deriveTopicId(feed);   // 66-hex-char topic ID; store/share this
```

> **Subscribers must use the identical descriptor the publisher used.** Because `region`, `owner`, `name`, and `write` are all hashed into the ID, dropping or changing any of them (e.g. omitting `write: 'owner'`) computes a *different* topic and you'll receive nothing.

---

4. Multiple publishers on one owned topic — *proposed, not yet implemented*

A common ask is "let the owner authorize additional publishers on a topic." As of v3.1.0 this is **not built**: `write: 'owner'` means exactly the one `owner` key.

It isn't a trivial add, because the set of writers can't live *in the descriptor* — the descriptor is hashed into the ID, so adding a writer would change the topic ID and orphan existing subscribers. The intended design (not yet shipped) is an **owner-signed writer roster**: the topic still commits to one `owner` key in its ID; the owner publishes a small signed record `{ topicId, writers: [authorId…], seq }`; storing nodes accept a publish if the signer is the owner **or** is in the latest roster the owner signed. Adding/removing a writer is just publishing a new roster — the topic ID never changes, and revocation is a `seq` bump. Treat this section as a roadmap item until it lands with its own tests.

---

5. Quick reference

	Open topic	Owned topic
Descriptor	`{ region, name }`	`{ region, owner, name, write: 'owner' }`
Who may publish	anyone (self-signed or anonymous)	only the `owner` key (node-enforced)
Who may subscribe	anyone	anyone given the descriptor / ID
Compute the ID without help?	yes (region + name)	no (needs the owner's Author ID)
`region` forms	name · `'0x89'` · `137`, or omit → node region	same
`signWith` on `pub`	an author, or `ANONYMOUS`	must be the `owner` author
Multiple writers	n/a	**proposed** (owner-signed roster) — not yet implemented

Two factories, one signer rule: `createNodeIdentity({lat,lng})` makes the connection identity; `createAuthorIdentity({persistAs?})` makes a location-free publish identity. Every signed publish names its signer with `signWith` (an author, or the `ANONYMOUS` sentinel) — there is no default signer.